Operating Systems Lab

Week 8: Page Replacement Algorithms

1. FIFO, LRU, and Optimal Replacement

Code:

```c
#include <stdio.h>
#include <stdbool.h>

#define MAX_PAGES 100

// display all frames
void printFrames(int frames[], int numFrames)
{
    printf("[");
    for (int i = 0; i < numFrames; i++)
        if (frames[i] != -1)
            printf("%d", frames[i]);
    printf("]\n");
}

// fifo page replacement
void simulateFIFO(int pages[], int numPages, int numFrames)
{
    printf("\nFIFO Page Replacement\n");

    int frames[numFrames]; // memory slots
    int pageFaults = 0;
    int next = 0;

    // init frames to -1
    for (int i = 0; i < numFrames; i++)
        frames[i] = -1;

    for (int i = 0; i < numPages; i++)
    {
        int currPage = pages[i];
        printf("Page %d -> ", currPage);

        bool present = false;
        for (int j = 0; j < numFrames; j++)
```

```
        {
            if (frames[j] == currPage)
            {
                present = true;
                break;
            }
        }

        if (!present)
        {
            frames[next] = currPage;
            next = (next + 1) % numFrames;
            pageFaults++;
        }

        printFrames(frames, numFrames);
    }
    printf("Total Page Faults (FIFO): %d\n", pageFaults);
}

// least recently used page replacement
void simulateLRU(int pages[], int numPages, int numFrames)
{
    printf("\nLRU Page Replacement\n");

    int frames[numFrames];
    int lastUsed[numFrames];
    int pageFaults = 0;

    for (int i = 0; i < numFrames; i++)
    {
        frames[i] = -1;
        lastUsed[i] = -1;
    }

    for (int i = 0; i < numPages; i++)
    {
        int currPage = pages[i];
        printf("Page %d -> ", currPage);

        // choosing suitable index
        int currIdx = -1;
        for (int j = 0; j < numFrames; j++)
        {
            if (frames[j] == currPage)
            {
```

```
                currIdx = j;
                break;
            }
        }

        if (currIdx != -1)
            lastUsed[currIdx] = i;
        else
        {
            pageFaults++;
            int emptySlot = -1;
            for (int j = 0; j < numFrames; j++)
            {
                if (frames[j] == -1)
                {
                    emptySlot = j;
                    break;
                }
            }

            if (emptySlot != -1)
            {
                frames[emptySlot] = currPage;
                lastUsed[emptySlot] = i;
            }
            else
            {
                int lruIdx = 0;
                int oldestTime = lastUsed[0];
                for (int j = 1; j < numFrames; j++)
                {
                    if (lastUsed[j] < oldestTime)
                    {
                        oldestTime = lastUsed[j];
                        lruIdx = j;
                    }
                }
                frames[lruIdx] = currPage;
                lastUsed[lruIdx] = i;
            }
        }
        printFrames(frames, numFrames);
    }
    printf("Total Page Faults (LRU): %d\n", pageFaults);
}
```

```c
// optimal page replacement
void simulateOptimal(int pages[], int numPages, int numFrames)
{
    printf("\nOptimal Page Replacement\n");

    int frames[numFrames];
    int pageFaults = 0;

    for (int i = 0; i < numFrames; i++)
        frames[i] = -1;

    for (int i = 0; i < numPages; i++)
    {
        int currPage = pages[i];
        printf("Page %d -> ", currPage);

        bool present = false;
        for (int j = 0; j < numFrames; j++)
        {
            if (frames[j] == currPage)
            {
                present = true;
                break;
            }
        }

        if (!present)
        {
            pageFaults++;
            int emptySlot = -1;
            for (int j = 0; j < numFrames; j++)
            {
                if (frames[j] == -1)
                {
                    emptySlot = j;
                    break;
                }
            }

            if (emptySlot != -1)
                frames[emptySlot] = currPage;
            else
            {
                int replaceIdx = -1;
                int farthestUse = -1;
```

```
                for (int j = 0; j < numFrames; j++)
                {
                    int nextUse = -1;
                    for (int k = i + 1; k < numPages; k++)
                    {
                        if (pages[k] == frames[j])
                        {
                            nextUse = k;
                            break;
                        }
                    }

                    if (nextUse == -1)
                    {
                        replaceIdx = j;
                        break;
                    }

                    if (nextUse > farthestUse)
                    {
                        farthestUse = nextUse;
                        replaceIdx = j;
                    }
                }
                frames[replaceIdx] = currPage;
            }
        }
        printFrames(frames, numFrames);
    }
    printf("Total Page Faults (Optimal): %d\n", pageFaults);
}

int main()
{
    printf("Page Replacement Algorithms\n");
    printf("Royden Miranda 1WA24CS240\n\n");

    int numPages, numFrames;
    int pages[MAX_PAGES];

    printf("Enter number of pages: ");
    if (scanf("%d", &numPages) != 1 || numPages <= 0)
        return 1;

    printf("Enter page reference string:\n");
    for (int i = 0; i < numPages; i++)
```

```
        if (scanf("%d", &pages[i]) != 1)
            return 1;

    printf("Enter number of frames: ");
    if (scanf("%d", &numFrames) != 1 || numFrames <= 0)
        return 1;

    simulateFIFO(pages, numPages, numFrames);
    simulateLRU(pages, numPages, numFrames);
    simulateOptimal(pages, numPages, numFrames);

    return 0;
}
```

Output:

```
Page Replacement Algorithms
Royden Miranda 1WA24CS240

Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

FIFO Page Replacement
Page 7 -> [7]
Page 0 -> [70]
Page 1 -> [701]
Page 2 -> [201]
Page 0 -> [201]
Page 3 -> [231]
Page 0 -> [230]
Page 4 -> [430]
Page 2 -> [420]
Page 3 -> [423]
Page 0 -> [023]
Page 3 -> [023]
Total Page Faults (FIFO): 10

LRU Page Replacement
Page 7 -> [7]
Page 0 -> [70]
Page 1 -> [701]
Page 2 -> [201]
Page 0 -> [201]
Page 3 -> [203]
Page 0 -> [203]
Page 4 -> [403]
Page 2 -> [402]
Page 3 -> [432]
Page 0 -> [032]
Page 3 -> [032]
Total Page Faults (LRU): 9

Optimal Page Replacement
Page 7 -> [7]
Page 0 -> [70]
Page 1 -> [701]
Page 2 -> [201]
Page 0 -> [201]
Page 3 -> [203]
Page 0 -> [203]
Page 4 -> [243]
Page 2 -> [243]
Page 3 -> [243]
Page 0 -> [043]
Page 3 -> [043]
Total Page Faults (Optimal): 7
```